1. ABSTRACT

Trading digital assets is a cornerstone of the modern digital economy, allowing participants to engage in dynamic financial markets. However, traditional paper trading applications face significant challenges, including latency issues, reliance on REST API polling, slippage misrepresentation, and paywalled access to complex order types. These issues often lead to unrealistic trading simulations and a lack of preparedness for live market volatility. To overcome these challenges, this project proposes a secure, low-latency, and highly accurate Crypto Trading Simulator by integrating a FastAPI asynchronous backend with live Binance WebSocket streams. This ensures real-time, zero-delay price ticks for accurate matching engine execution. Users can manage a virtual wallet to trade BTC, ETH, SOL, and SHIB using virtual USD. The implementation of an active background task evaluates resting orders (Market, Limit, Stop Market, Stop Limit, and Trailing Stops) against live market data. By combining a robust ReactJS frontend featuring TradingView's lightweight charts with a high-frequency Python matching engine, the proposed system aims to provide a professional-grade, free, and hyper-realistic educational sandbox for trading strategies.

2. INTRODUCTION

2.1 Context and Challenge

Traditional and basic electronic paper trading systems are burdened by issues like delayed price updates, polling latency, and restricted feature sets. This lack of realistic execution environments undermines the educational value of simulated trading. The integration of high-performance WebSockets for real-time data ingestion and an asynchronous matching engine offers a promising solution. This fusion aims to establish an accurate, low-latency, and efficient trading simulator.

Current free trading simulators lack complete accuracy and are highly susceptible to execution delays (e.g., polling latency and slippage). This vulnerability erodes the realism of the simulation. A critical need exists for a system that can accurately mirror live market conditions, execute complex conditional orders, and ensure the integrity of the virtual wallet ledger in real-time.

3. LITERATURE REVIEW

A Literature Review should provide a clear, organized, and critical overview of existing research related to your project topic. It shows what is already known, what methods have been used, and where gaps still exist.

3.1 What is Already Known?

The feasibility of integrating web-based simulators for market education is well established. Researchers and financial platforms have proposed frameworks that use historical or delayed data to mimic market movements. It is a recognized tool for teaching basic market mechanics (e.g., the difference between bids and asks) without risking real capital. Real-time data streaming via WebSockets is widely acknowledged in the literature as the industry standard for high-frequency trading platforms.

3.2 Methods Used

Existing literature primarily relies on:

- **System Architecture Design:** Proposing traditional Client-Server architectures utilizing REST APIs for fetching periodic price updates.
- **Basic State Management:** Evaluating simple database updates where a "buy" button deducts simulated cash and credits a simulated asset based on a static price point.
- **Comparative Studies:** Analyzing the performance differences between basic line charts and complex candlestick rendering in DOM vs. Canvas environments.

3.3 Where Gaps Still Exist (The Project's Niche)

While the components exist, a gap remains in a fully developed, integrated system that specifically focuses on:

- **Robust Asynchronous Integration:** Many educational simulators use synchronous polling; the detailed, accurate, and low-latency integration of WebSockets into a background matching loop needs focused development.
- **Optimized Hybrid System:** There is a need for a demonstrable, end-to-end system that perfectly balances the high-speed requirements of a trading engine (evaluating Trailing Stops in-memory) with the strict ACID compliance of a relational database for wallet management

4. TECHNOLOGIES

In a project synopsis, the Technologies section should clearly list and explain the tools, platforms, and technical resources that will be used to develop, analyze, or implement the project. It helps the reader understand the technical foundation of your work.

I. Frontend: User Interface

The trading terminal and user dashboard will be built using the following:

- **ReactJS:** The core library for building the dynamic and responsive user interface, handling real-time status updates.
- **Tailwind CSS:** A utility-first framework providing highly responsive and customizable styling for the application interface.
- **TradingView Lightweight Charts:** A lightweight and modern HTML5 canvas library that allows the frontend to securely and smoothly render high-performance financial visualizations.

II. Core Logic and Data Layer: Backend & Matching Engine

This layer ensures the execution and integrity of the simulated market.

- **FastAPI (Python):** The high-performance framework used to write the core routing and order logic, utilizing Python's asyncio for background tasks.
- **WebSockets:** The protocol utilized to connect directly to the Binance Public API, providing a continuous, unauthenticated stream of live tick data for BTC, ETH, SOL, and SHIB.
- **Order Matching Engine:** Automated asynchronous loops that enforce market logic: tracking asset prices, accepting pending orders, and autonomously executing trades when conditions are met.

III. Database Tier

This secure layer handles the critical wallet and ledger checks.

- **PostgreSQL:** A dedicated, secure repository for storing the virtual wallet balances, order history, and user authentication credentials with strict row-level locking to prevent race conditions.
- **Redis (Upcoming):** An in-memory data store specialized for capturing and evaluating dynamically moving Trailing Stop thresholds before persisting them to the main database.

IV. Development and Deployment Tools

These tools facilitate the secure and efficient creation, testing, and deployment of the application.

- **Git/GitHub:** Version control system used for collaborative development, code management, and tracking changes across the frontend and backend codebase.
- **Uvicorn:** A lightning-fast ASGI server implementation used to run the FastAPI asynchronous application.
- **Docker:** Used to containerize the application environments, ensuring parity between local testing and production deployment

5. PROJECT REQUIREMENT

Project requirements include all the essential resources needed to develop the project. These consist of functional requirements (the main features the system must perform) and non-functional requirements (performance, security, and usability needs). It also includes hardware requirements like computers or devices, and software requirements such as programming languages, frameworks, libraries, and databases. These requirements ensure the project runs smoothly and meets its objectives.
Following are the minimum hardware and software requirements for building this project:

Hardware Requirements

Component	**Specification**
Processor	Intel core i5-8250U @ 1.60GHz (or equivalent)
RAM	8 GB or 16GB and more
Storage	10 GB Free Space Minimum

Software Requirements

Component	**Specification**
Operating System	LINUX, WINDOWS 10, 11
Programming Languages	Python, JavaScript/TypeScript
Database	PostgreSQL, SQLite
Web Browser	Chrome, Microsoft Edge, Safari, Mozilla Firefox
Text Editor	Visual Studio Code, Sublime Text Editor

Other Software	Postman, Docker
Documentation	Microsoft Word, LaTeX/WeasyPrint

6. PROBLEM STATEMENT

6.1 Problem Description

Existing paper trading methods are highly vulnerable to various forms of latency and architectural shortcuts, leading to results that lack realism. The specific issues are:

1. **Slippage and Latency:** Current HTTP polling methods are inadequate, causing the simulator to execute trades at prices the market has already moved past.
2. **Lack of Advanced Mechanics:** Simulators struggle to process dynamically changing variables, meaning essential risk-management tools like Trailing Stops are omitted.
3. **Concurrency Failures:** Poorly designed database logic in simulators can lead to race conditions where users "double-spend" virtual funds.

6.2 Impact

Inaccurate execution and missing order types directly threaten the learning process of new traders. When users test strategies in an environment that does not accurately reflect real-world constraints, they develop a false sense of security, ultimately leading to significant capital loss when transitioning to live markets.

6.3 Need for a Solution

There is a critical and immediate need for a novel e-trading simulator that can simultaneously guarantee:

1. **Zero-Latency Tick Processing:** To ensure the execution engine reacts instantly to wick movements in the market.
2. **Robust Matching Logic:** To guarantee that complex conditional orders (Stop Limits) execute exactly as they would on a centralized exchange.
3. **Atomic Ledger Integrity:** To allow concurrent order processing without compromising the mathematical accuracy of the user's virtual wallet

7. OBJECTIVE

1. To design a realistic electronic trading system by integrating live WebSocket feeds with a Python asynchronous backend.
2. To implement a secure virtual wallet system with row-level database locking to enforce strict balance management.
3. To develop a background matching loop that evaluates Market, Limit, and Stop orders against live tick data autonomously.
4. To deploy a ReactJS frontend featuring professional-grade candlestick charts powered by TradingView.
5. To automate the tracking of dynamic Trailing Stop thresholds using in-memory architecture (Redis) to preserve system speed.

8. METHODOLOGY

8.1. Defining and Designing the System

This initial stage sets the foundation for the entire project.

- **Requirement Analysis:** The first step involves Project Definition, which entails clearly defining project objectives, capturing detailed user needs (for UI/UX), and finalizing the scope.
- **Design:** Creating detailed flowcharts for the data pipeline, showing how tick data flows from Binance -> FastAPI WebSocket Task -> Matching Engine -> PostgreSQL.

8.2. Core Engine Development (Backend)

This phase implements the non-negotiable execution features of the system.

- **FastAPI Module:** Developing the REST endpoints for wallet funding and order placement.
- **Asynchronous Loop:** Writing the asyncio task that continuously consumes the Binance !ticker@arr stream and checks resting database orders against the new prices.
- **Secure Data Module:** Implementing the SQLAlchemy ORM layer to ensure all executed trades trigger atomic updates to the user's USD and Crypto asset balances.

8.3. Application and Interface Development

This phase focuses on the user-facing and connecting components.

- **ReactJS Dashboard:** Building the responsive UI to accept user input for 5 distinct order types.
- **Charting Module:** Integrating lightweight-charts and feeding it directly via a localized client-side WebSocket to ensure 60fps rendering of market data independent of the backend load.

8.4. Testing, Deployment, and Maintenance

The final stages ensure system readiness and ongoing stability.

- **System Testing:** Rigorous testing of the matching logic using pytest, simulating high-volatility market scenarios to ensure Limit orders do not execute prematurely.

- **Stress Testing:** Utilizing tools like locust to bombard the FastAPI server with concurrent order requests to validate database lock integrity.
- **Maintenance & Updates:** Evolving the architecture to support Redis for complex Trailing Stops and expanding the asset pair offerings

9. REFERENCES

1. FastAPI Documentation. High-performance ASGI framework. Available at: https://fastapi.tiangolo.com/
2. Python Software Foundation. asyncio — Asynchronous I/O. Available at: https://docs.python.org/3/library/asyncio.html
3. ReactJS Foundation. React Documentation. Available at: https://reactjs.org/
4. TradingView Lightweight Charts Library. Available at: https://tradingview.github.io/lightweight-charts/
5. Binance API Official Documentation. Public WebSocket Streams. Available at: https://binance-docs.github.io/apidocs/
6. SQLAlchemy 2.0 Documentation. Core and ORM Operations. Available at: https://docs.sqlalchemy.org/en/20/